Path ORAM: Design Document

Advanced Topics in Online Privacy and Cybersecurity (67515)
Spring 2026 , Assignment 1

Mohammad Awawda

1. Architecture Overview

This document describes the design and implementation of a non-recursive Path ORAM system. The system consists of two main components: a Server that stores encrypted data in a binary tree structure, and a Client that manages access patterns, encryption, and integrity verification. The server is oblivious to the data content and the client's access patterns.

1.1 Server Architecture

The server stores data in a complete binary tree implemented as an array. This design enables O(1) parent/child lookups: for node at index i, its children are at indices 2i and 2i+1, and its parent is at i//2. The tree has at least N leaves, where N is the number of data blocks supported (rounded up to the next power of 2).

Each node in the tree is a bucket containing exactly Z = 4 slots. Each slot holds one encrypted block (either a real data block or a dummy block). After encryption, real and dummy blocks are indistinguishable to the server. The server exposes two main operations:

- **read_path(leaf)**: Returns all buckets on the path from the given leaf to the root.
- **write_path(path_data)**: Overwrites the buckets along a path with new encrypted data.

The server never performs encryption or decryption. It operates purely on opaque ciphertexts and has no knowledge of block IDs, data contents, or which blocks are real versus dummy.

1.2 Client Architecture

The client is the trusted component that manages all cryptographic operations and maintains the following local state:

Component	Type	Size	Purpose
Encryption Key	AES-256 key	32 bytes	Encrypt/decrypt all blocks
Position Map	Dict: id → leaf	O(N)	Maps each block to a random leaf
Version Hash	Dict: id → SHA-256	O(N)	Integrity verification (rollback)
Stash	List of (id, data)	O(log N) expected	Overflow buffer

1.3 Core Access Operation

All three public operations are implemented through a single internal _access method. This ensures that every operation is structurally identical from the server's perspective, preserving obliviousness. The _access method performs the following steps:

Step 1 : Lookup and Reassign: The client looks up the block's assigned leaf in the position map. If the block is new, a random leaf is assigned. Immediately, the block is reassigned to a fresh random leaf. This ensures consecutive accesses to the same block traverse different paths.

Step 2 : Read Path: The client reads the entire root-to-leaf path from the server (using the original leaf from Step 1). All blocks on this path are decrypted, and real blocks are moved into the client's stash.

Step 3 : Operate on Stash: The requested operation is performed on the stash: for a read, the block's data is retrieved; for a write, the block's data is updated; for a delete, the block is removed.

Step 4 : Write Back (Eviction): The client writes the path back to the server. Processing buckets from leaf to root, it greedily fills each bucket with blocks from the stash whose assigned leaf is compatible with that bucket. Remaining slots are filled with encrypted dummies. All blocks are re-encrypted with fresh nonces.

The leaf-to-root eviction order is critical: it pushes blocks as deep into the tree as possible, keeping upper buckets (especially the root) less congested and minimizing stash growth.

2. Security Design

2.1 End-to-End Encryption

All data stored on the server is encrypted using AES-256. This is an authenticated encryption scheme that provides both confidentiality and integrity in a single operation. Each encryption uses a fresh 12-byte random nonce, ensuring that encrypting the same plaintext twice produces completely different ciphertexts(non-deterministic encryptor).

Each encrypted block is structured as the nonce (12 bytes), followed by the GCM tag (16 bytes), followed by the actual ciphertext. The ciphertext itself starts with 4 bytes for the block ID (encoded as a big-endian integer), then the B bytes of data. For dummy blocks, we just set the ID to -1 and pad the data with zeros (once everything is encrypted), the server has no way to tell which blocks are real and which are dummies.

Every write-back operation re-encrypts all blocks on the path (including unchanged blocks and dummies) with fresh nonces. This prevents the server from detecting which blocks were modified, which remained the same, and which are dummy versus real.

2.2 Data Integrity and Authentication

Data integrity is ensured at two levels:

Tamper detection: AES-GCM produces a 16-byte authentication tag alongside each ciphertext. On decryption, the tag is verified; if even a single bit has been modified, decryption fails and the client raises an integrity violation error. This protects against a malicious server modifying stored ciphertexts.

Rollback detection: The client maintains a local version hash (SHA-256) for each stored block. After decrypting a block, the client computes its hash and compares it to the stored hash. If the hashes differ, the server has returned stale data (a rollback attack). This costs only 32 bytes of additional client-side storage per block.

2.3 Access Pattern Obliviousness

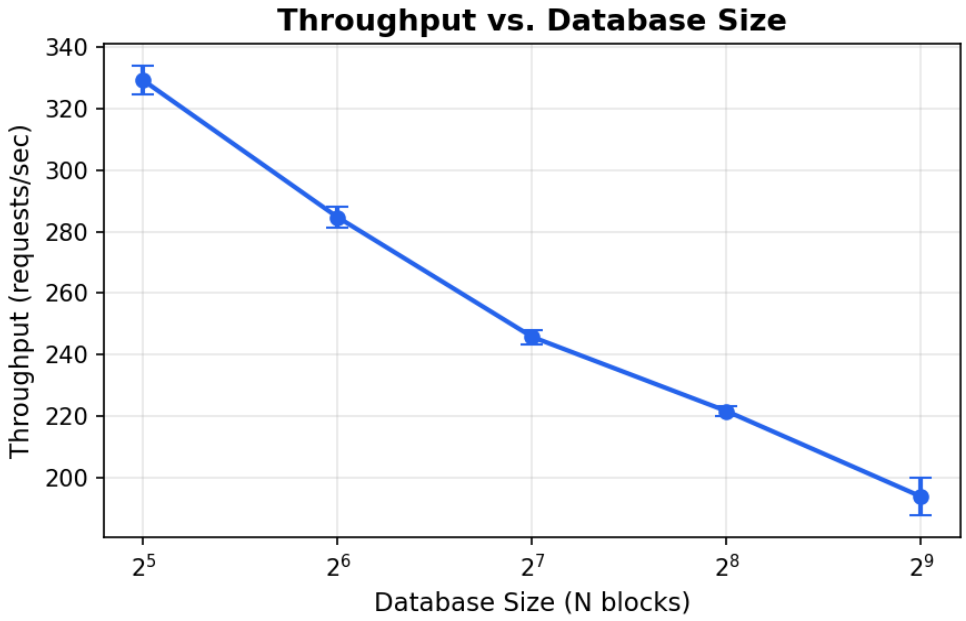
Every operation looks identical to the server: one full path read followed by one full path write, where the leaf is chosen uniformly at random (from the server’s perspective, since the position map is local). Combined with re-encryption using fresh nonces, the server cannot distinguish between operation types, cannot tell which block was accessed, and cannot link consecutive accesses to the same block.

3. Performance Benchmarks

All benchmarks were averaged over 3 independent runs to reduce noise. The experiments were conducted on a single-threaded Python implementation. Below we report the results along with a discussion of observed trends.

3.1 Throughput (requests/sec) vs. N (Database Size)

N (blocks)	Throughput (req/sec)	Std Dev
32	329.2	±4.6
64	284.6	±3.4
128	245.7	±2.4
256	221.6	±1.7
512	193.8	±6.0

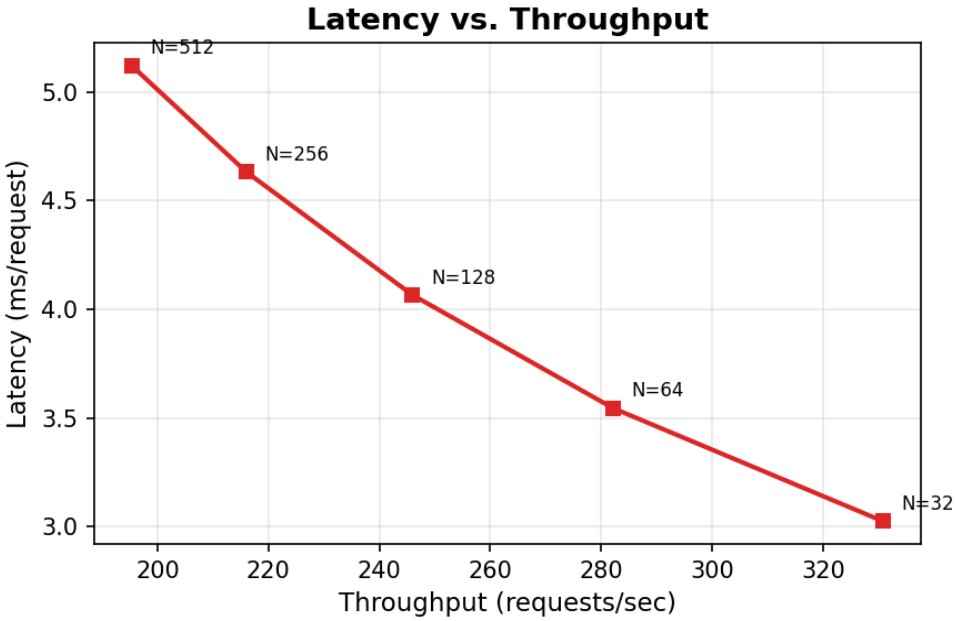


Discussion: Throughput decreases as N increases. This is expected because a larger N means more leaves, hence a taller tree (more levels = $\log_2$ N,since N increases). Each access reads and writes one full root-to-leaf path, so the number of buckets processed per operation grows logarithmically with N. The overhead per operation includes Z × ($\log_2$ N + 1) encryption and decryption operations, which dominate the runtime.

3.2 Latency (ms/request) vs. Throughput

N	Latency (ms/req)	Throughput (req/sec)
32	3.025	330.8
64	3.545	282.2
128	4.066	246.0

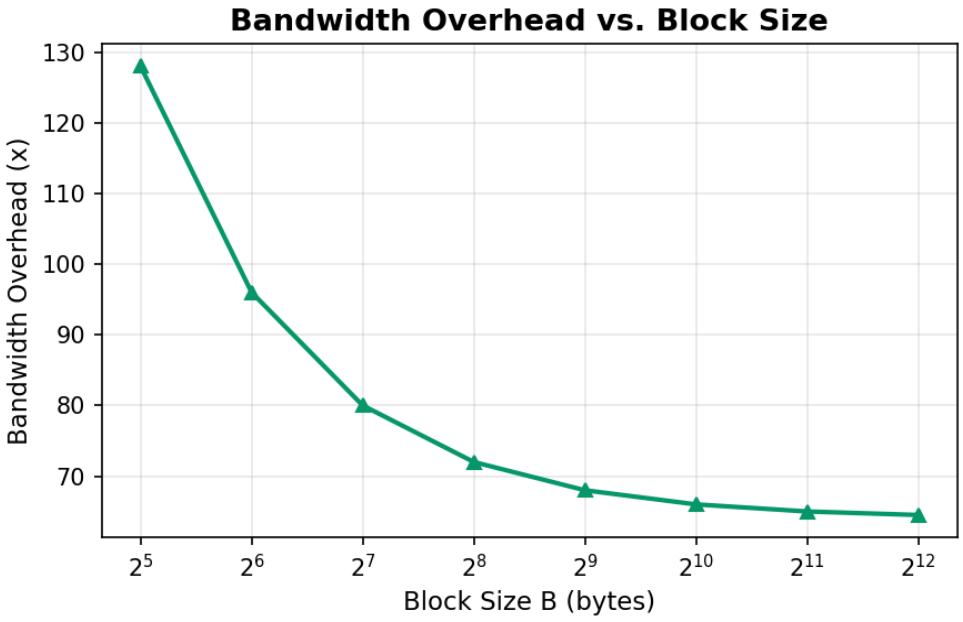
256	4.633	216.0
512	5.121	195.3



Discussion: Latency and throughput exhibit an inverse relationship, as expected. As N grows, path length increases, each operation takes longer (higher latency), and fewer operations complete per second (lower throughput). The relationship is approximately linear because both metrics scale with log N which is the path length. Doubling N shifts the operating point to higher latency and lower throughput, confirming the O(log N) per-access cost of Path ORAM.

3.3 Bandwidth Overhead vs. Block Size (B)

Block Size B (bytes)	Bandwidth Overhead (x)
32	128.00
64	96.00
128	80.00
256	72.00
512	68.00
1024	66.00
2048	65.00
4096	64.50

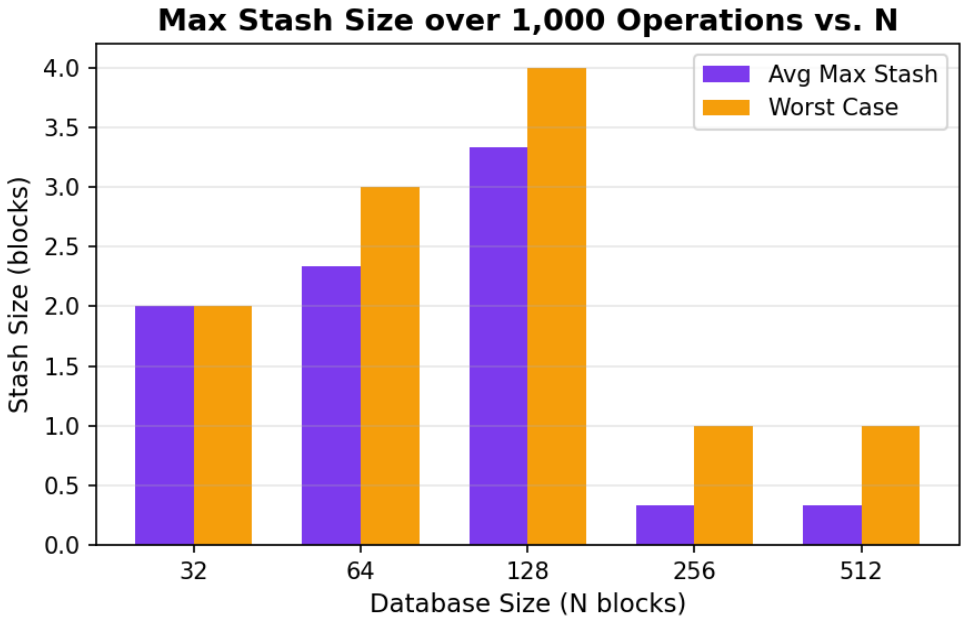


Discussion: The bandwidth overhead goes down as we increase the block size B. This makes sense, every access reads and writes a full path, which means 2 × (L+1) × Z blocks get transferred regardless of how big or small each block is. On top of the actual data, each encrypted block carries 32 bytes of fixed overhead (12 for the nonce, 16 for the tag, and 4 for the block ID). When B is small, like 32 bytes, that fixed overhead basically doubles the size of each block, so the bandwidth overhead is huge. But as B gets larger, those extra 32 bytes become a tiny fraction of the total, and the overhead settles down to roughly 2 × (L+1) × Z , which is just the inherent cost of Path ORAM always reading and writing entire paths of full buckets**.**

3.4 Maximum Stash Size vs. N

N (blocks)	Avg Max Stash	Worst Case
32	2.0	2
64	2.3	3

128	3.3	4
256	0.3	1
512	0.3	1



Discussion: The stash stayed really small across all our tests , at most 4 blocks, even after 1,000 operations in a row. The greedy eviction from leaf to root does a good job pushing blocks back into the tree, so they don't pile up in the stash.

4. Design Questions

4.1 Does the Implementation Benefit from Multicore?

The current implementation does not benefit from multicore processing. Each ORAM access is inherently sequential: the client must read a path, modify the stash, and write the path back before the next access can begin, because the stash is shared mutable state. Within a single access, there is limited parallelism opportunity: the Z × (L+1) encryption/decryption operations within one path could theoretically be parallelized, but we should check for python's overhead for doing so, and if it'd be beneficial in this case.

4.2 Threat Modeling

Honest-but-Curious Server: Against an honest-but-curious server (one that follows the protocol correctly but tries to learn information from observations), our implementation provides strong protection. The server observes only:

- encrypted blocks that are indistinguishable from random data,
- which path (leaf) is accessed each operation , but since leaves are reassigned randomly, consecutive accesses to the same block go to different paths, and
- the timing and frequency of operations , but all operations look identical (same path length, same number of blocks read/written). The server cannot determine which block is being accessed, whether the operation is a read or write, or whether the same block is accessed twice.

Fully Malicious Server: Against a fully malicious, our implementation provides partial protection:

- **Tamper resistance (protected):** AES authentication tags ensure that any modification to stored ciphertexts is detected upon decryption. The client will raise an integrity error if the server modifies any block.
- **Rollback attacks (protected):** The client stores a SHA-256 hash of each block's current data. After decrypting, the hash is verified. This detects rollback attacks where the server serves a stale version of a specific block.
- **Full-tree rollback (vulnerable):** If the server rolls back the entire tree to a previous state, and the client's local position map and version hashes are also somehow reverted, the current design cannot detect this. correction: the client could maintain a tree over the server's entire storage, with the root hash stored locally and updated on every access. Alternatively, a monotonic counter stored in trusted hardware could detect rollbacks.
- **Denial of service (vulnerable):** The server can always refuse to respond or return garbage. This is an inherent limitation of any remote storage system and cannot be solved by ORAM alone.